

AWS CLOUD SUPPORT PROJECT

Hands-on Cloud Infrastructure & Troubleshooting Lab

AWS VPC • ALB • Auto Scaling • S3 • IAM • CloudWatch • SNS

Portfolio / GitHub Documentation

1. Project Overview

This project is a hands-on AWS Cloud Support and infrastructure troubleshooting lab. The environment was designed to practice networking, load balancing, high availability, IAM-based access, object storage, monitoring, alerting, and real incident troubleshooting.

- Region used: ap-south-1
- Custom VPC with public and private subnets
- Internet-facing Application Load Balancer (ALB)
- Auto Scaling Group (ASG) with desired/min/max capacity of 2/2/2
- S3 bucket access from EC2 through an IAM role
- CloudWatch CPU monitoring with SNS email notification
- Intentional troubleshooting incidents followed by verification and cleanup

2. Project Objectives

- Build a realistic small-scale AWS environment using core Cloud Support services.
- Understand how VPC networking affects EC2 connectivity.
- Configure an ALB and target group for application traffic.
- Verify Auto Scaling replacement behavior.
- Use an IAM role instead of hard-coded AWS credentials for EC2-to-S3 access.
- Create a CloudWatch alarm and verify SNS notification during a CPU incident.
- Practice troubleshooting from symptoms to root cause and verification.
- Clean up resources to avoid unnecessary AWS charges.

3. Architecture

The architecture consists of a custom VPC containing public and private networking components. The ALB receives HTTP traffic and forwards it to healthy EC2 targets. The Auto Scaling Group maintains two application instances. EC2 instances use an IAM role to access S3. CloudWatch monitors CPU utilization and sends an alert through SNS when the configured threshold is exceeded.

Component	Configuration / Role	Purpose
VPC	cloud-support-vpc, 10.0.0.0/16	Isolated project network
Subnets	2 public + 1 private	Network segmentation
Internet Gateway	cloud-support-igw	Internet connectivity for public subnet
EC2	Web server instances	Host the web application
ALB	cloud-support-lb-hu-re	Distribute HTTP traffic
Target Group	web-tg	Track healthy application targets
ASG	cloud-support-as	Maintain desired instance capacity
S3	Project bucket + lab-test-file.txt	Object storage and access test
IAM	ec2-s3-read-role	Controlled EC2-to-S3 access
CloudWatch	web-server-high-cpu	CPU monitoring and alarm
SNS	cloud-support-alerts	Email notification

4. VPC & Networking Setup

- Created a custom VPC named cloud-support-vpc with CIDR 10.0.0.0/16.
- Created two public subnets across two Availability Zones and one private subnet.
- Configured an Internet Gateway named cloud-support-igw.
- Configured a public route table with 0.0.0.0/0 routed through the Internet Gateway.
- Associated the required public subnet with the public route table.
- Verified that incorrect route-table association was the cause of an EC2 Instance Connect failure.

5. Application Load Balancer

- Created internet-facing ALB: cloud-support-lb-hu-re.
- Configured HTTP listener on port 80.
- Created target group: web-tg.
- Registered web server targets and verified their health status.
- Tested traffic distribution and verified that service remained available when one target was removed while another healthy target remained.

6. Auto Scaling

- Created launch configuration/template for the web application.
- Configured ASG cloud-support-as with desired/min/max capacity of 2/2/2.
- Attached the ASG to the ALB target group.
- Intentionally terminated an ASG-managed instance.
- Verified that Auto Scaling automatically launched a replacement instance to restore desired capacity.

7. S3 + IAM Role Integration

- Created an S3 bucket and uploaded lab-test-file.txt for the access test.
- Created IAM role ec2-s3-read-role with read-only S3 permissions.
- Attached the role to the EC2 instance.
- Verified S3 access from the EC2 terminal using AWS CLI.
- Used aws configure list and STS identity checks to understand that the instance was using its IAM role rather than hard-coded access keys.

```
aws s3 ls
aws s3 ls s3://<bucket-name>
aws configure list
aws sts get-caller-identity
```

8. CloudWatch + SNS Monitoring

- Created CloudWatch CPU alarm web-server-high-cpu.
- Configured the project alarm around a 60% CPU threshold.
- Configured SNS topic cloud-support-alerts with email notification.
- Generated CPU load intentionally to simulate an infrastructure incident.
- Observed the alarm transition to In Alarm and verified receipt of the SNS email notification.

9. Troubleshooting Cases

Case 1 — EC2 Instance Connect Failure

Symptom	EC2 Instance Connect could not establish the expected connection.
Root cause	The public subnet was not associated with the intended public route table.
Fix / Verification	Associated the public subnet with public-rt and verified connectivity again.

Case 2 — ALB Target Health

Symptom	Target health/routing required verification during ALB testing.
Root cause	Target registration/health state needed to be checked before treating the load balancer as healthy.
Fix / Verification	Verified target group health and tested the ALB only after healthy targets were available.

Case 3 — ASG Instance Failure

Symptom	One ASG-managed instance was intentionally terminated.
Root cause	Simulated instance failure.
Fix / Verification	No manual replacement was required; ASG launched a replacement and restored desired capacity to 2.

Case 4 — EC2 to S3 Access

Symptom	S3 access had to be validated without storing access keys on the server.
Root cause	Access depended on the attached IAM role and its permissions.
Fix / Verification	Verified the role identity and successfully listed the S3 bucket/object through AWS CLI.

Case 5 — High CPU Alert

Symptom	CPU utilization crossed the configured threshold.
Root cause	Intentional CPU stress test.
Fix / Verification	CloudWatch detected the spike, alarm entered In Alarm, and SNS delivered the email notification.

10. Evidence / Screenshots

Selected AWS Console screenshots from the lab session, covering ALB networking, Auto Scaling, launch template configuration, and CloudWatch CPU monitoring. Account identifiers, resource IDs and other private details have been masked.

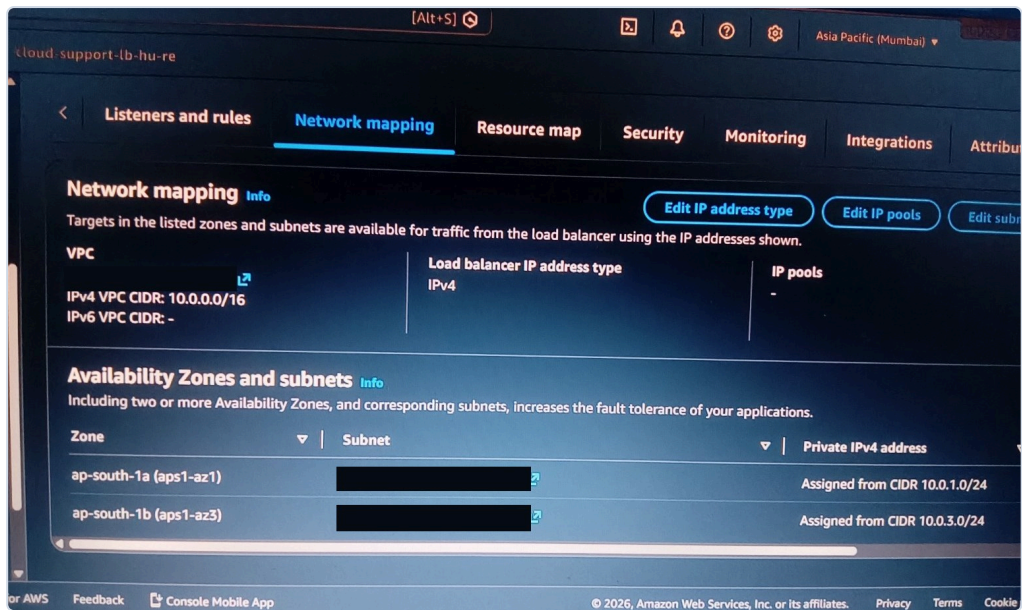


Figure 1 — ALB network mapping (VPC CIDR and two-AZ subnet coverage)

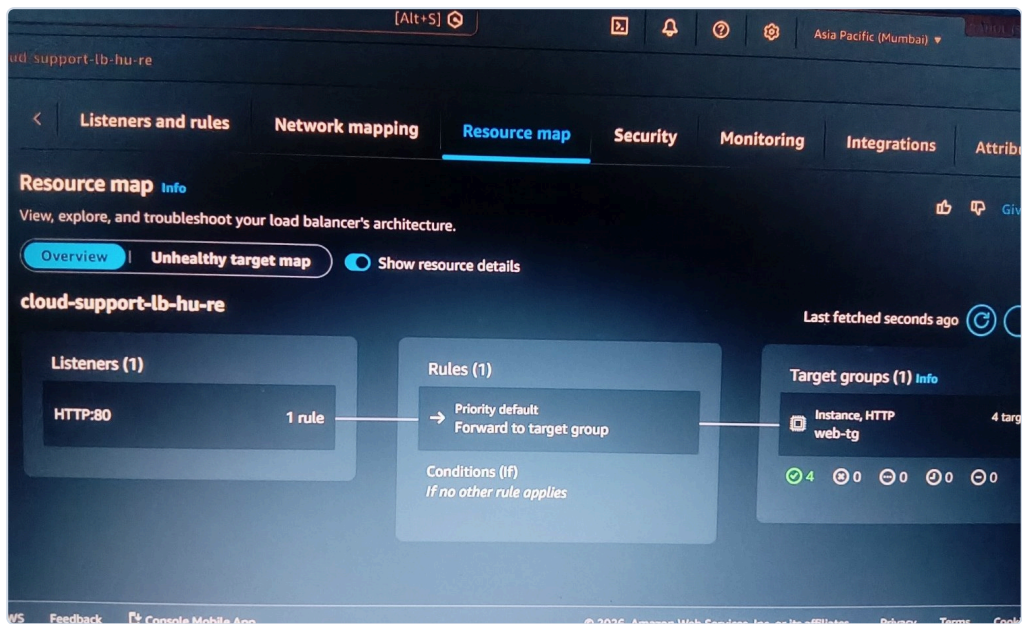


Figure 2 — ALB resource map: HTTP:80 listener → rule → target group web-tg (4 healthy targets shown)

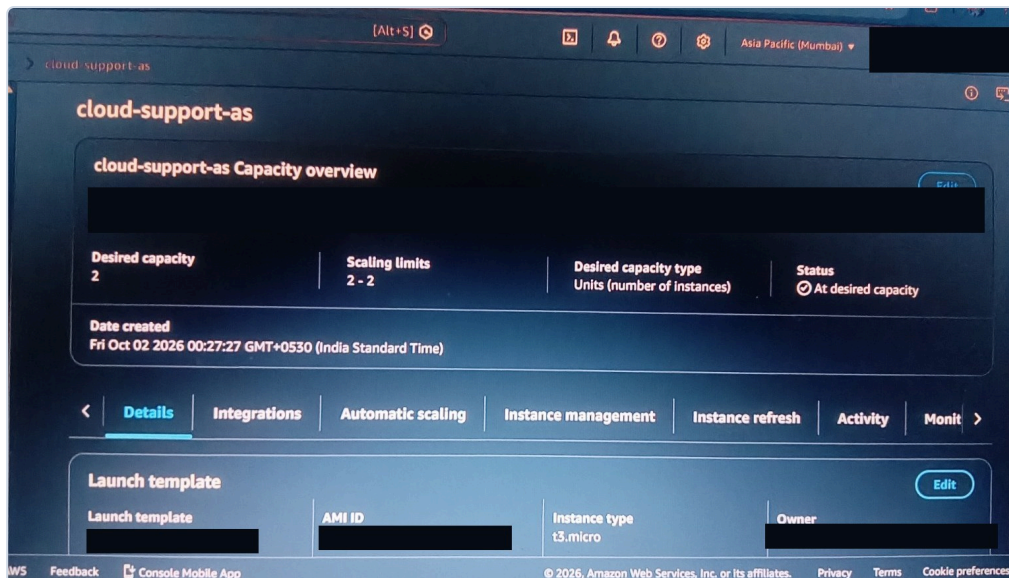


Figure 3 — Auto Scaling Group capacity overview (desired 2, limits 2-2, t3.micro)

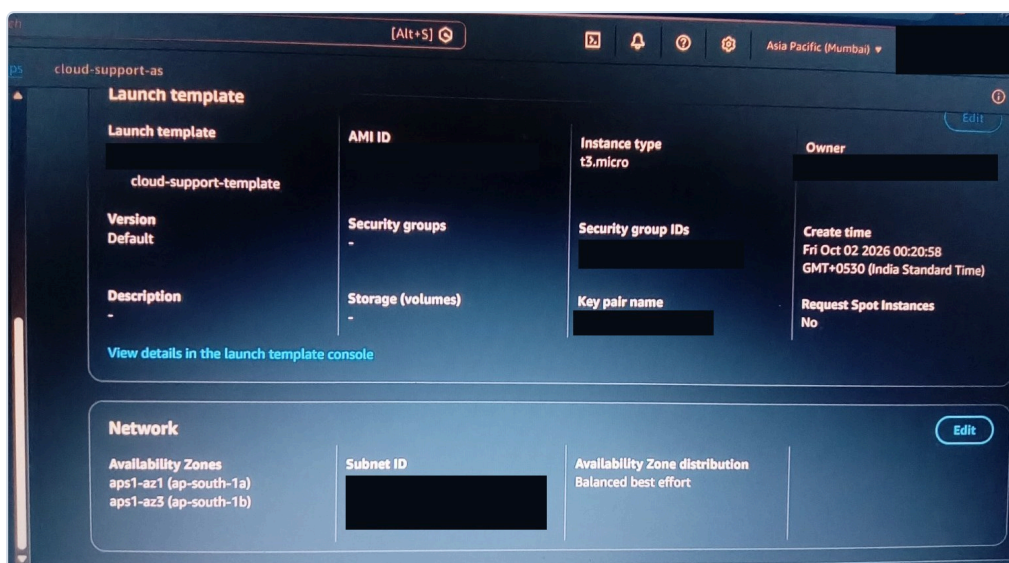


Figure 4 — Launch template and ASG network configuration

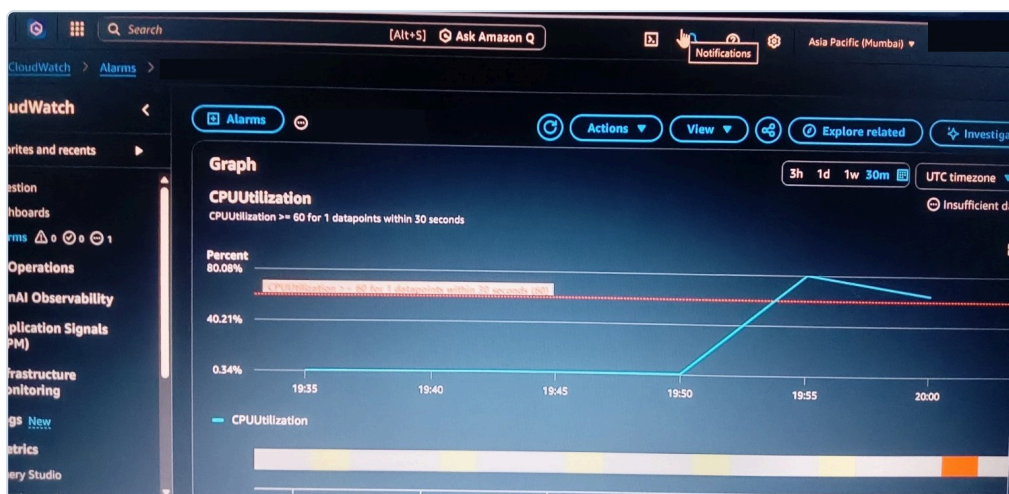


Figure 5 — CloudWatch CPUUtilization alarm graph (threshold 60%)

11. Security & Cost-Control Practices

- Used IAM roles for EC2-to-S3 access instead of embedding long-lived access keys.
- Used read-only S3 permission for the demonstrated access requirement.
- Avoided publishing AWS credentials, private keys, passwords, SMTP credentials, or unnecessary account identifiers.
- Performed a full cleanup after the lab.
- Deleted project EC2 instances, ALB, target group, ASG, launch template, AMI/snapshot artifacts, CloudWatch alarm, SNS topic, S3 resources, IAM role, and custom networking resources.

12. Project Scope & Limitations

This is a hands-on learning and portfolio project, not a production deployment. The integrated lab documented here focuses on VPC, ALB, Auto Scaling, S3, IAM, CloudWatch and SNS.

- RDS was not part of this integrated lab.
- NAT Gateway was not created as part of this integrated lab.
- A public Route 53 domain mapping was not part of this integrated lab.
- The project demonstrates Cloud Support troubleshooting concepts rather than claiming production readiness.

13. Key Learnings

- AWS networking problems can often be isolated by checking subnet, route table, Internet Gateway, security group and target health in order.
- ALB health checks are essential before diagnosing application availability.
- Auto Scaling can recover from instance failure automatically when desired capacity is configured correctly.
- IAM roles provide a safer way for EC2 workloads to access AWS services.
- CloudWatch + SNS provides a practical monitoring-to-notification workflow.
- Troubleshooting should follow a repeatable flow: symptom → evidence → root cause → fix → verification.

14. Conclusion

This project demonstrates practical Cloud Support skills using a small but realistic AWS environment. It combines infrastructure configuration with incident simulation and troubleshooting rather than only showing service setup.